LANGGRAPH RESEARCH KIT · 브레인스토밍

# LangGraph 필터 설계 브레인스토밍

2026-11-11 · 작성: 대표 + Claude · 상태: 브레인스토밍 초안

섹션 1

## 목적과 핵심 질문

핵심은 하나다. **용도별로 끼워 넣는 필터가 실제로 위험을 막는지, 그 대가(과차단·비용)는 얼마인지를 숫자로 보여 주고, 그 숫자로 필터를 계속 고친다.** 그래프 자체는 공통이고, 상품의 차이는 필터와 그 검증 기록에서 나온다.

답해야 할 질문:

1. **막는가:** 도메인마다 알려진 환각 유형(없는 조문, 단정 진단, 출처 없는 수치 등)을 필터가 몇 %나 잡는가?
2. **너무 막지는 않는가:** 정상 답변을 잘못 막거나 재작성시키는 비율은? 막기만 하는 필터는 쓸 수 없다.
3. **얼마나 드는가:** 필터와 루프가 더하는 토큰·시간은? 검색으로 줄인 입력과 비교해 남는가?
4. **나아지고 있는가:** 규칙 버전이 바뀔 때마다 같은 문항에서 1~3이 어떻게 움직였는가?
5. **믿을 만한가:** 표본 수와 구간을 같이 볼 때도 같은 결론인가? 비교용으로 고른 설정이 검증용에서도 버티는가?

대상 도메인은 네 개다: **리서치, 법률, 의료, 생활**(지역 행정·생활 법률·소비자 정보처럼 일상에서 묻는 것). 결과물은 두 갈래다: 내가 튜닝하고 결정하는 데 쓰는 자료, 외부에 보여 주고 신뢰를 얻는 자료.

섹션 2

## 설계 원칙

여섯 가지를 고정하고, 나머지는 실험으로 정한다.

1. **바로 답하게 두지 않는다.** 모든 요청은 분류 → 로컬 검색 → 규정 필터를 거친다. 이 파이프라인이 해자다.
2. **그래프는 고정, 필터는 교체.** 도메인이 바뀌어도 노드와 엣지는 그대로고, `check_input` / `verify` / `finalize` 세 자리와 State 추가 칸만 바꾼다.
3. **통제는 엣지와 State에서.** 반복·비용·오류 판정은 순수 Python 경로 함수가 한다. 모델에게 "그만할지"를 묻지 않는다.
4. **안전 지표는 품질과 섞지 않는다.** 평균 97%여도 단정 진단 1건이면 의료 필터는 불합격이다. 보드 맨 위에 따로, Wilson 상한과 함께 보인다.
5. **검증 안 된 글은 내보내지 않는다.** 고칠 수 있으면 예산 안에서 다시, 아니면 부분 보고서 + "확인 못 한 항목". 지어내지 않는다.
6. **주장은 측정으로.** "단일 LLM보다 낫다"는 가설이다. 합성 문항에서 측정하고, 실제 모델 결과와 가짜 경로 결과를 같은 표에 섞지 않는다.

섹션 3

## 아이디어 풀

지금까지 나온 아이디어를 다듬어 범주로 묶었다. 상태: ✅ 구현(규칙 기반) · 🔄 오늘 밤 큐 · 💡 아이디어.

| 범주 | 아이디어 | 필터 검증과의 관계 | 상태 |
|---|---|---|---|
| 구조 | 바깥 레일: classify → rag → 비용·반복 관문, RAG는 독립 노드 | 필터를 끼울 자리(screen·verify)를 만든다 | ✅ |
| 구조 | 분업 노드: 정제 → 추출 → 찬반·조건 분리 → 비교 → 작성 → 검수 | 단계별로 어디서 틀렸는지 추적된다 | ✅ |
| State | 리듀서로 주장·수치·관계 누적, 모든 조각에 출처 id + 원문 위치 | 검수가 "무엇과 대조할지"가 생긴다 | ✅ |
| State | 도메인 칸: 법률 시차 로그, 의료 금기 목록, 생활 서류 체크리스트 | 도메인별 검수 규칙의 재료 | 🔄 법률·의료 뼈대 |
| 루프 | 표적 루프: 지표 누락은 그 지표만 재검색, 문장 문제는 재작성만 | 검수 코드가 곧 되돌아갈 주소 | ✅ |
| 루프 | 진전 없으면 중단, 코드별 우선순위, 노드별 토큰 상한 | 필터가 비용 폭탄을 만들지 않게 | 🔄 R25 |

| 범주 | 아이디어 | 필터 검증과의 관계 | 상태 |
|---|---|---|---|
| 비용 | 그림자 토큰 회계, State 압축 프롬프트, 자료 규모별 손익분기 | 필터의 비용을 숫자로 | ✅ |
| LangGraph | 병렬 분기(Send), 문서별 map-reduce 추출 | 더 많은 근거를 같은 시간에 | 🔄 R6·R28 |
| LangGraph | 체크포인트 + interrupt(사람 확인 후 재개) | 고위험 도메인의 사람 승인 관문 | 🔄 R13 |
| LangGraph | 도메인별 서브그래프 | 필터를 통째로 갈아 끼우기 | 💡 |
| 모델 | 계획·추출·작성 구조화 출력, 로컬 모델, 규칙 폴백 | 규칙 대 모델 필터의 비교 대상 | 🔄 R10·R11·R27 |
| 모델 | LLM 검수자(judge)를 규칙 검수 옆에 두고 불일치만 사람에게 | 규칙의 놓침과 과차단을 찾는 레이더 | 💡 |
| 모델 | 자기일관성: 같은 질문 3회 생성 후 공통 주장만 채택 | 불확실한 주장을 필터 전에 걸러냄 | 💡 |
| 모델 | 모르면 모른다고 답하기(abstain)를 명시적 경로로 | "억지 답변"을 필터가 막는 대신 설계로 줄임 | 💡 |

섹션 4

## 도메인별 필터 설계안

네 도메인 모두 같은 그래프에 끼우고, 아래 칸만 바꾼다. 리서치는 규칙 v1이 돌고, 법률·의료는 오늘 밤 뼈대, 생활은 설계 단계다.

| 항목 | 리서치 | 법률 | 의료 | 생활 |
|---|---|---|---|---|
| 주된 환각 | 인과 유실, 일반론, 출처 없는 수치 | 없는·폐지 조문, 시차 뒤섞임 | 단정 진단, 금기 누락, 용량 수치 | 지역 예외 무시, 구버전 양식·기준 |
| check_input | 범위·길이 | 사건 유형 분기(형사·민사·가사) | **응급 분류 → 즉시 차단·안내** | 관할(시·군·구)·신청 유형 분기 |
| State 추가 칸 | 주장·관계·수치 | 시간순 사건 로그(리듀서) | 증상·복용 약·금기 목록 | 구비서류 체크리스트·시행일 |
| 검색 강제 | 로컬 문서, 찬반·조건 분리 | 검증된 조문 DB만 | 임상 가이드라인 근거 | 최신 규정 DB만 |
| verify 코드 | UNSUPPORTED_NUMBER, CAUSAL_LOSS, CONSTRAINT_DROPPED, ONE_SIDED, LOW_DENSITY, GAP | UNSUPPORTED_STATUTE, REPEALED_CITED, TIMELINE_MIXED | DEFINITIVE_DIAGNOSIS, CONTRAINDICATION_DROPPED, UNSUPPORTED_DOSE | OUTDATED_FORM, LOCAL_EXCEPTION_DROPPED, CHECKLIST_INCOMPLETE |
| 재시도 불가(안전) | NO_EVIDENCE | 폐지 조문 인용 | 단정 진단, 응급 | 구버전 양식 |
| finalize | 그대로 | 법률 자문 아님 고지 | 면책 + 전문의 상담 권고 | 담당 기관 확인 안내 |
| 사람 승인(interrupt) | 선택(지표 계획) | 권장(사건 유형) | **필수** | 선택 |
| 상태 | ✅ 규칙 v1 | 🔄 뼈대(R18·R22) | 🔄 뼈대(R23·R32) | 🔄 뼈대(R30) |

**생활** 도메인은 "로컬 비즈니스·행정"에 생활 법률·소비자 정보까지 넓힌 범위로 본다. 공통점은 "어디서, 언제 기준인가"가 답을 바꾼다는 것이다.

섹션 5

## 필터 성능 검증 방법

필터 하나를 네 가지 숫자로 판정한다: **잡는가(검출률), 너무 막는가(과차단률), 남기는가(보존률), 얼마나 드는가(비용)**. 평균 점수 하나로 묶지 않는다.

| 지표 | 무엇을 재나 | 어떻게 재나 | 목표 예시 |
|---|---|---|---|
| 오류 주입 검출률 | 알려진 환각을 필터가 잡는 비율 | 정답 보고서에 일부러 결함을 넣어(수치 바꾸기, 조건 빼기, 가짜 조문, 단정 표현) verify에 통과시켜 본다. 결함 종류별로 따로 집계 | 안전 결함 ≥ 99% (Wilson 하한까지) |

| 지표 | 무엇을 재나 | 어떻게 재나 | 목표 예시 |
|---|---|---|---|
| 과차단률 | 정상 답변을 잘못 막거나 재작성시키는 비율 | 결함 없는 정답 보고서를 통과시켜 걸리는 비율. 바꿔 말하기·동의어 변형도 포함 | ≤ 5% |
| 보존률 | 필수 항목(지표·인과·조건·반대 근거)이 최종 보고서에 남는 비율 | 지금 보드의 품질 지표 | 구간과 함께 보고 |
| 비용 | 필터·루프가 더하는 토큰·시간 | 필터 켬/끔 비교, 자료 규모별 손익분기 | 기준선 대비 배수 |

**문항 세트 세 가지**(도메인마다)

1. **골든셋:** 합성 문서 + 정답(필수 항목, 금지 항목). compare / confirm으로 나눠 튜닝은 compare에서만.
2. **오류 주입셋:** 골든 정답에서 결함 변형을 자동 생성. 사람이 쓰지 않아도 수백 건이 나오고, 필터 규칙을 바꿀 때마다 다시 돌린다.
3. **적대셋:** 주입 공격, 애매한 질문, 경계 사례(응급인지 애매한 증상, 개정 직후 조문).

**판정 방식**

- 규칙 필터와 LLM 검수자를 나란히 돌려 **둘이 갈리는 사례만** 사람에게 보여 준다. 사람 판정이 가장 비싼 자원이다.
- 비율에는 항상 표본 수와 Wilson 95% 구간. 0건은 "없음"이 아니라 "상한 x%"로 쓴다.
- 필터를 끄고 켠 **절제(ablation)** 비교를 기본으로 넣는다. 필터가 없을 때와 비교해야 필터의 값이 보인다.

섹션 6

## 피드백 루프

보드에서 본 문제가 다음 규칙 버전과 회귀 시험으로 돌아오는 한 바퀴를 하루 단위로 돈다.

1. **실행:** 밤샘에 도메인별 골든·오류 주입·적대셋을 돌리고 결과를 기록(JSONL)에 쌓는다.
2. **보기:** 보드 맨 위 안전 지표 → 과차단률 → 보존률 → 비용 순서로 본다. 규칙과 LLM 검수자가 갈린 사례만 "판정 대기"에 모인다.
3. **판정:** 사례마다 나는 세 가지 중 하나만 고른다. 필터가 맞음 / 필터가 놓침 / 필터가 과차단. 이유는 한 줄.
4. **규칙 수정:** 판정을 모아 Codex 명세로 만들고, 규칙 파일을 고치면 `version`을 올린다.
5. **고정:** 판정한 사례는 그대로 회귀 시험이 된다. 다음 버전이 같은 실수를 다시 하면 시험이 깨진다.
6. **비교:** 보드의 실행 기록에서 이전 버전 대비 1~4번 숫자가 어떻게 움직였는지 본다. 나빠진 숫자가 있으면 그 버전은 기본값이 되지 못한다.

판정은 보드 화면에서 바로 하는 것이 목표다. 다른 사람과 판정을 공유해 저장하려면 게시된 페이지의 공유 저장 기능을 쓰는 방안을 검토한다.

섹션 7

## 실험 자료 패키지

같은 실행 기록에서 두 갈래 자료를 만든다. 내부용은 결정을 위한 것이라 자세하고, 외부용은 신뢰를 위한 것이라 한계를 먼저 적는다.

| 자료 | 대상 | 담는 것 | 형태 |
|---|---|---|---|
| 튜닝 보드 | 나 | 설정·손잡이별 검출률·과차단률·보존률·비용, 판정 대기 목록 | 로컬 HTML 보드(매일 갱신) |
| 주간 실험 노트 | 나 | 바꾼 규칙, 숫자 변화, 버린 아이디어와 이유 | 문서 1장 |
| 결함 카탈로그 | 나 + 외부 | 도메인별 환각 유형과 예시(합성), 필터가 잡는지 | 표 + 예시 |
| 필터 성적표 | 외부 | 도메인별 4개 숫자 + 표본 수·구간, 버전, 한계 | 한 장 PDF(다크) |
| 시연 보드 | 외부 | 합성 문항으로 기준선 vs 그래프 나란히, 손잡이 슬라이더 | 공개 링크 페이지 |
| 사례 스토리 3편 | 외부 | 리서치·법률·의료 각 1편: 질문 → 단일 호출의 실수 → 필터가 잡은 지점 → 최종 보고서 | 슬라이드 또는 문서 |

| 자료 | 대상 | 담는 것 | 형태 |
|---|---|---|---|
| 도입 가이드 | 외부(개발자) | 키트를 다른 프로젝트에 끼우는 법, 필터 교체, 비용 상한 | README + 예제 |

외부용 자료의 규칙: 합성 데이터만, 실제 모델과 규칙 경로 결과를 구분 표시, "보장" 대신 "구간".

모든 PDF는 다크 모드가 기본이다.

섹션 8

## 우선순위와 다음 실험 백로그

검증 도구를 먼저, 필터를 나중에 늘린다. 재는 방법이 없으면 필터가 좋아졌는지 판단할 수 없기 때문이다.

- ☐ **오류 주입기:** 골든 정답에서 결함 변형(수치·조건·인과·인용·단정 표현)을 자동 생성하는 도구. 도메인 공통.
- ☐ **네 가지 숫자 보드:** 검출률·과차단률·보존률·비용을 도메인별로, 안전 지표는 맨 위에 따로(R21·R31을 확장).
- ☐ **필터 끄기/켜기 절제 비교**를 보드 기본 표시로.
- ☐ **판정 대기 목록 + 세 버튼 판정**(맞음·놓침·과차단)과 판정의 회귀 시험 자동 변환.
- ☐ **LLM 검수자 병행**(로컬 모델 또는 마스킹 후 허용된 모델)으로 규칙과의 불일치 수집.
- ☐ **법률·의료 골든셋 30건씩**과 적대셋, 생활 도메인 뼈대와 문항 10건.
- ☐ **실제 모델 기준선:** 같은 문항으로 로컬 Llama 단일 요약 vs 그래프.
- ☐ **필터 성적표 PDF(다크)**와 시연 보드 공개 링크 첫판.

추천 순서: 오류 주입기 → 네 가지 숫자 보드 → 법률·의료 골든셋 → 판정 루프 → LLM 검수자 → 실제 모델 기준선 → 외부 자료.

섹션 9

## 열린 질문

- 생활 도메인의 범위: 지역 행정·생활 법률·소비자 정보 중 어디부터 시작할까?
- 안전 지표 목표(예: 검출 ≥ 99%, 과차단 ≤ 5%)를 도메인별로 다르게 둘까? 의료는 과차단을 더 허용할까?
- LLM 검수자는 로컬 모델만? 마스킹 후 외부 모델을 평가 용도에 한해 허용할까?
- 판정을 나 혼자 할까, 도메인 전문가(변호사·의사) 1명씩 검토를 받을까?
- 외부 공개 시점: 리서치 성적표만 먼저 낼까, 법률·의료까지 모아서 낼까?
- 필터를 상품으로 팔 때 단위: 도메인 팩 단위 라이선스인가, 튜닝 보드 포함 서비스인가?

섹션 10

## 계측 보드: 4대 지표와 이벤트 스키마

도메인 파이프라인(3~5개)을 데이터로 튜닝(Data-Driven Harness Tuning)하려면 보드가 내부 상태·루프 회전수·토큰 원가·환각 차단을 실시간으로 보여 줘야 한다. 단위는 Spring Boot — FastAPI — LangGraph 3단 백엔드를 관통하는 `run_id` 이벤트 하나.

| 패널 | 측정 | 튜닝 활용 |
|---|---|---|
| 1. 실행 궤적 | 경로 retrieve→grade→rewrite…, 종료 유형 END / give_up / cutoff | 어떤 질문에서 rewrite 루프가 많은지 보고 검색·채점 기준 조정 |
| 2. 비용 통제 | 모델별 호출 수(SLM 심판 vs Flagship 작문), 입력/출력 토큰, 건당 원가(원) | 목표 원가(예: 1.5원) 감시, Fast/Deep Track 라우팅 비율 조절 |
| 3. 환각 방어 | RelevanceGrade 탈락률, GroundedGrade 반려 사유 로그 | 도메인별 수치·조문 오류 진단, 채점기 과차단 점검 |
| 4. 레이턴시 | 노드별 ms(검색·abatch 채점·생성), TTFT | 병목 노드 식별 → 병렬화·경량 모델 교체 |

> 전체 스키마: `docs/METRICS_BOARD.ko.md`

LANGGRAPH RESEARCH KIT · BRAINSTORM

# LangGraph filter design brainstorm

2026-11-11 · Authors: owner + Claude · Status: brainstorm draft

SECTION 1

## Purpose and key questions

One goal: **show in numbers whether a filter plugged in for a given use actually blocks the risks, what it costs (over-blocking, tokens), and use those numbers to keep improving the filter.** The graph is shared; what makes a product different is the filter and its validation record.

Questions to answer:

1. **Does it catch?** What share of each domain's known hallucination types (non-existent articles, definitive diagnoses, numbers without a source) does the filter catch?
2. **Does it over-block?** How often does it wrongly block or force a rewrite of a correct answer? A filter that only blocks is useless.
3. **What does it cost?** How many tokens and seconds do the filter and its loops add, and does retrieval still save more than that?
4. **Is it improving?** As rule versions change, how do 1–3 move on the same cases?
5. **Can we trust it?** Does the conclusion hold with sample sizes and intervals in view, and does a config chosen on the compare set hold on the confirm set?

Four domains: **research, legal, medical, everyday life** (local administration, everyday law, consumer information). Two kinds of output: material I use to tune and decide, and material that shows outsiders why to trust it.

SECTION 2

## Design principles

Six fixed points; everything else is decided by experiment.

1. **Never let the model answer straight away.** Every request goes route → local retrieval → rule filter. That pipeline is the moat.
2. **The graph is fixed, the filter is swapped.** Nodes and edges stay; only `check_input` / `verify` / `finalize` and a few extra State fields change per domain.
3. **Control lives in edges and State.** Pure-Python route functions decide loops, cost and errors. The model is never asked whether to stop.
4. **Safety metrics are never averaged into quality.** 97% average with one definitive diagnosis fails a medical filter. Safety goes on top of the board, separately, with a Wilson upper bound.
5. **Unverified text is never returned.** Fixable → retry within budget; otherwise a partial report plus a "not verified" list. Nothing is invented.
6. **Claims are measured.** "Better than a single LLM" is a hypothesis, measured on synthetic cases; real-model results and rule-path results never share a table.

SECTION 3

## Idea pool

Ideas so far, refined and grouped. Status: ✅ implemented (rule-based) · 🔄 in tonight's queue · 💡 idea.

| Area | Idea | Link to filter validation | Status |
|---|---|---|---|
| Structure | Outer rail: classify → rag → cost/loop gates; RAG is one isolated node | creates the slots (screen, verify) where filters plug in | ✅ |
| Structure | Division of labour: refine → extract → split pro/con/limits → compare → write → verify | shows at which step something went wrong | ✅ |
| State | Reducers accumulate claims, metrics, relations; every piece has source id + character span | gives the verifier something to check against | ✅ |
| State | Domain fields: legal timeline log, medical contraindication list, everyday document checklist | material for domain verify rules | 🔄 legal, medical skeletons |
| Loops | Targeted loops: missing metric → re-search that metric only; prose issue → rewrite only | verify codes are return addresses | ✅ |
| Loops | Stop when no progress, per-code priority, per-node token caps | keeps filters from causing cost blow-ups | 🔄 R25 |
| Cost | Shadow-token accounting, State-compressed prompts, corpus-size break-even | puts the filter's cost in numbers | ✅ |
| LangGraph | Parallel branches (Send), per-document map-reduce extraction | more evidence in the same time | 🔄 R6, R28 |
| LangGraph | Checkpoints + interrupt (resume after human review) | human approval gate for high-risk domains | 🔄 R13 |
| LangGraph | Per-domain subgraphs | swap a whole filter in one piece | 💡 |
| Models | Structured output for plan/extract/write, local models, rule fallback | rule vs model filters to compare | 🔄 R10, R11, R27 |
| Models | LLM judge beside the rule verifier; only disagreements go to a human | radar for the rules' misses and over-blocks | 💡 |
| Models | Self-consistency: generate 3 times, keep only shared claims | filters uncertain claims before the filter | 💡 |
| Models | Abstain ("I don't know") as an explicit path | reduces forced answers by design instead of blocking them | 💡 |

SECTION 4

## Domain filter designs

All four plug into the same graph; only these cells change. Research runs rule v1; legal and medical get skeletons tonight; everyday life is in design.

| Item | Research | Legal | Medical | Everyday life |
|---|---|---|---|---|
| Main hallucination | lost causality, generic text, numbers without a source | non-existent or repealed articles, mixed-up timelines | definitive diagnosis, missing contraindications, doses | regional exceptions ignored, outdated forms and rules |
| check_input | scope, length | case-type branch (criminal, civil, family) | **emergency triage → stop and direct to care** | jurisdiction and application-type branch |
| Extra State fields | claims, relations, metrics | chronological event log (reducer) | symptoms, current medication, contraindications | document checklist, effective dates |
| Forced retrieval | local documents, pro/con/limits split | verified statute DB only | clinical guideline evidence | latest-rule DB only |

| Item | Research | Legal | Medical | Everyday life |
|---|---|---|---|---|
| verify codes | UNSUPPORTED_NUMBER, CAUSAL_LOSS, CONSTRAINT_DROPPED, ONE_SIDED, LOW_DENSITY, GAP | UNSUPPORTED_STATUTE, REPEALED_CITED, TIMELINE_MIXED | DEFINITIVE_DIAGNOSIS, CONTRAINDICATION_DROPPED, UNSUPPORTED_DOSE | OUTDATED_FORM, LOCAL_EXCEPTION_DROPPED, CHECKLIST_INCOMPLETE |
| Not retryable (safety) | NO_EVIDENCE | repealed article cited | definitive diagnosis, emergency | outdated form |
| finalize | unchanged | "not legal advice" notice | disclaimer + see a specialist | check with the agency |
| Human approval (interrupt) | optional (metric plan) | recommended (case type) | **required** | optional |
| Status | ✅ rule v1 | 🔄 skeleton (R18, R22) | 🔄 skeleton (R23, R32) | 🔄 skeleton (R30) |

**Everyday life** widens "local business and administration" to everyday law and consumer information. What they share: where and as of when decides the answer.

SECTION 5

## How to validate a filter

Judge each filter by four numbers: **does it catch (detection), does it over-block (false-block rate), does it keep what matters (retention), what does it cost**. Never one blended score.

| Metric | Measures | How | Example target |
|---|---|---|---|
| Seeded-fault detection | share of known hallucinations the filter catches | inject faults into gold reports (change a number, drop a condition, fake an article, definitive wording) and run verify; count per fault type | safety faults ≥ 99% (at the Wilson lower bound) |
| False-block rate | share of correct answers blocked or sent to rewrite | run fault-free gold reports, including paraphrases and synonyms | ≤ 5% |
| Retention | share of must-keep items (metrics, causality, conditions, counter-evidence) in the final report | today's board quality metric | report with intervals |
| Cost | tokens and time added by filters and loops | filter on/off; corpus-size break-even | multiple of baseline |

**Three case sets** (per domain)

1. **Gold set:** synthetic documents + answers (must-have and must-not items). Split compare / confirm; tune on compare only.
2. **Seeded-fault set:** fault variants generated automatically from gold answers. Hundreds of cases without hand-writing; rerun on every rule change.
3. **Adversarial set:** injection attacks, ambiguous questions, edge cases (symptoms that may or may not be an emergency, articles amended last week).

**Judging**

- Run the rule filter and an LLM judge side by side; **only their disagreements** go to a human. Human judgement is the scarcest resource.
- Every rate carries its sample size and Wilson 95% interval. Zero is written "upper bound x%", not "none".
- **Ablation** (filter off vs on) is a default view. A filter's value only shows against its absence.

SECTION 6

## Feedback loop

One daily cycle takes a problem seen on the board back into the next rule version and a regression test.

1. **Run:** overnight, run each domain's gold, seeded-fault and adversarial sets; append results (JSONL).
2. **Look:** on the board, safety → false blocks → retention → cost, in that order. Cases where the rules and the LLM judge disagree collect in a review queue.
3. **Judge:** for each case I pick one of three: filter right / filter missed / filter over-blocked, with a one-line reason.
4. **Change rules:** judgements become Codex specs; a rule-file change bumps `version`.
5. **Pin:** every judged case becomes a regression test; repeating the mistake breaks the test.
6. **Compare:** the run history shows how 1–4 moved against the previous version; a version that worsens any of them does not become the default.

The aim is to judge on the board itself. Sharing judgements with others would use the published page's shared storage.

SECTION 7

## Experiment material

Two kinds of material from the same run history. Internal material is detailed because it drives decisions; external material leads with limits because it earns trust.

| Material | For | Contents | Form |
|---|---|---|---|
| Tuning board | me | detection, false blocks, retention, cost per config and knob; review queue | local HTML board, updated daily |
| Weekly experiment note | me | rules changed, numbers moved, ideas dropped and why | one page |
| Fault catalogue | me + outside | hallucination types per domain with synthetic examples; whether the filter catches them | table + examples |
| Filter scorecard | outside | the four numbers per domain + sample sizes, intervals, version, limits | one-page PDF (dark) |
| Demo board | outside | baseline vs graph side by side on synthetic cases, knob sliders | public link page |
| Three case stories | outside | research, legal, medical: question → what a single call got wrong → where the filter caught it → final report | slides or doc |
| Integration guide | outside (developers) | plugging the kit into another project, swapping filters, cost caps | README + examples |

Rules for external material: synthetic data only, real-model vs rule-path results labelled, intervals instead of guarantees.

All PDFs default to dark mode.

SECTION 8

## Priorities and next experiments

Build the measuring tools first, add filters second: without a way to measure, you cannot tell whether a filter improved.

- ☐ **Fault injector:** generates fault variants (numbers, conditions, causality, citations, definitive wording) from gold answers; shared by all domains.
- ☐ **Four-number board:** detection, false blocks, retention, cost per domain, safety on top (extends R21, R31).
- ☐ **Filter on/off ablation** as a default board view.

- ☐ **Review queue + three-button judging** (right, missed, over-blocked), judgements turned into regression tests automatically.
- ☐ **LLM judge in parallel** (local model, or an allowed model on masked text) to collect disagreements with the rules.
- ☐ **Legal and medical gold sets of 30 each** plus adversarial sets; everyday-life skeleton with 10 cases.
- ☐ **Real-model baseline:** local Llama one-shot summary vs the graph on the same cases.
- ☐ **Filter scorecard PDF (dark)** and a first public demo board link.

Suggested order: fault injector → four-number board → legal/medical gold sets → judging loop → LLM judge → real-model baseline → external material.

SECTION 9

## Open questions

- Everyday-life scope: start with local administration, everyday law or consumer information?
- Different safety targets per domain (e.g. detection ≥ 99%, false blocks ≤ 5%)? Should medical tolerate more over-blocking?
- LLM judge: local models only, or outside models on masked text for evaluation only?
- Judging alone, or one domain expert (lawyer, physician) reviewing each?
- Going public: research scorecard first, or wait for legal and medical?
- Selling filters: licence per domain pack, or a service that includes the tuning board?

SECTION 10

## Metrics board: four panels and an event schema

To tune 3–5 domain pipelines from data (Data-Driven Harness Tuning), the board must show internal state, loop counts, token cost and hallucination blocks live. The unit is one `run_id` event across the Spring Boot — FastAPI — LangGraph stack.

| Panel | Measured | Used for |
|---|---|---|
| 1. Workflow trace | Path retrieve→grade→rewrite…; end type END / give_up / cutoff | See which questions loop in rewrite; tune retrieval and grading |
| 2. Cost control | Calls per model (SLM judge vs flagship writer), input/output tokens, cost per run (KRW) | Watch the cost target (e.g. 1.5 KRW); set the Fast/Deep Track routing ratio |
| 3. Guardrail audit | RelevanceGrade drop rate, GroundedGrade rejection reasons | Diagnose number/article errors per domain; catch over-strict graders |
| 4. Latency | Per-node ms (retrieve, abatch grading, generate), TTFT | Find the bottleneck; parallelise or swap in a lighter model |

Full schema: `docs/METRICS_BOARD.en.md`